INSTITUT TEKNOLOGI DEL

MATERI TUGAS

Pemrograman Berorientasi Objek

SEMESTER GANJIL T.A 2026/2027

Tanggal / Sesi	:	Jumat, 9 Oktober 2026
Semester	:	Ganjil
Mata Kuliah	:	Pemrograman Berorientasi Objek
Minggu/Sesi	:	05/ 03
Aktivitas	:	Polymorphism & Abstract Class
Durasi	:	110 Menit
Setoran	:	Softfile
Batas Pengumpulan	:	Jumat, 9 Oktober 2026 Pukul 23.59
Tujuan	:	- Menerapkan polymorphism dalam studi kasus kompleks - Mendesain hierarchy class dengan abstract class - Menggunakan method overriding dan overloading secara efektif - Membuat sistem yang extensible dengan OOP principles

Petualangan di Dunia Sihir Harry Potter dengan Polymorphism & Abstract Class

BAGIAN 1: MEMAHAMI KONSEP DASAR (25 menit)

Bagian 1: Polymorphism (30 menit)

1.1 Method Overloading: Mantra dengan Berbagai Efek

Dalam dunia sihir, sebuah mantra bisa memiliki efek yang berbeda tergantung cara mengucapkannya. Misalnya, mantra "*Expelliarmus*" bisa digunakan untuk melucuti senjata, tapi jika diucapkan dengan kekuatan lebih, bisa membuat lawan terlempar.

Instruksi:

1. Jalankan kode di bawah dan amati outputnya.
2. Tambahkan overload method **`ucapkan()`** yang menerima parameter **`String namaMantra`** dan **`String target (tanpa parameter kekuatan)`**.
3. Di dalam method, cetak efek yang sesuai.

Pertanyaan:

- Apa yang terjadi jika kita menghapus salah satu method overloading? Misalnya, method dengan dua parameter.
- Bisakah kita membuat method overloading dengan urutan parameter yang berbeda? Coba buat method **`ucapkan(int kekuatan, String namaMantra)`.**

Kita akan membuat File: **Mantra.java** yang memiliki kelas **`Mantra`** yang memiliki method **`ucapkan()`** yang di-overload.

```
import java.util.*;

public class Mantra {
    // Method ucapkan() dengan satu parameter
    public void ucapkan(String namaMantra) {
        System.out.println("Mengucapkan mantra: " + namaMantra);
    }

    // Overload: method ucapkan() dengan dua parameter
    public void ucapkan(String namaMantra, int kekuatan) {
        System.out.println("Mengucapkan mantra: " + namaMantra + " dengan kekuatan " + kekuatan);
        if (kekuatan > 10) {
            System.out.println("Efek: Lawan terlempar!");
        } else {
            System.out.println("Efek: Senjata terlepas.");
        }
    }

    // Overload: method ucapkan() dengan tiga parameter
    public void ucapkan(String namaMantra, int kekuatan, String target) {
        System.out.println("Mengucapkan mantra: " + namaMantra + " dengan kekuatan " + kekuatan + " pada " + target);
        if (kekuatan > 10) {
            System.out.println("Efek: " + target + " terlempar!");
        } else {
            System.out.println("Efek: Senjata " + target + " terlepas.");
        }
    }

    public static void main(String[] args) {
        Mantra mantra = new Mantra();
        mantra.ucapkan("Expelliarmus");
        mantra.ucapkan("Expelliarmus", 5);
        mantra.ucapkan("Expelliarmus", 15, "Voldemort");
    }
}
```

Output:

```
Mengucapkan mantra: Expelliarmus
Mengucapkan mantra: Expelliarmus dengan kekuatan 5
Efek: Senjata terlepas.
Mengucapkan mantra: Expelliarmus dengan kekuatan 15 pada Voldemort
Efek: Voldemort terlempar!
```

1.2 Method Overriding: Mantra yang Diwariskan

Kita akan membuat hierarki penyihir. Setiap penyihir memiliki cara yang berbeda dalam mengucapkan mantra.

Instruksi:

1. Jalankan kode di bawah.
2. Tambahkan kelas `RonWeasley` yang mewarisi `Penyihir` dan override method `ucapkanMantra()` dengan implementasi yang sesuai untuk Ron.

Pertanyaan:

- Apa yang terjadi jika kita tidak menggunakan annotation @Override? Apakah kode masih berjalan?
- Apa keuntungan menggunakan polymorphism dalam contoh ini?

Kita akan membuat File: **DemoPenyihir.java** yang di dalamnya terdapat kelas `Penyihir`, `HarryPotter`, dan `Hermonie` yang memiliki method yang di-overload.

```java
import java.util.*;

class Penyihir {
    public void ucapkanMantra() {
        System.out.println("Mengucapkan mantra dengan cara biasa.");
    }
}

class HarryPotter extends Penyihir {
    @Override
    public void ucapkanMantra() {
        System.out.println("Harry Potter mengucapkan mantra dengan tongkatnya yang unik.");
    }
}

class Hermione extends Penyihir {
    @Override
    public void ucapkanMantra() {
        System.out.println("Hermione mengucapkan mantra dengan tepat dan percaya diri.");
    }
}

public class DemoPenyihir {
    public static void main(String[] args) {
        Penyihir penyihir1 = new HarryPotter();
        Penyihir penyihir2 = new Hermione();

        penyihir1.ucapkanMantra(); // Output: Harry Potter mengucapkan mantra dengan tongkatnya yang unik.
        penyihir2.ucapkanMantra(); // Output: Hermione mengucapkan mantra dengan tepat dan percaya diri.
    }
}
```

Output:

```
Harry Potter mengucapkan mantra dengan tongkatnya yang unik.
Hermione mengucapkan mantra dengan tepat dan percaya diri.
```

Bagian 2: Abstract Class (30 menit)

2.1 Membuat Abstract Class: Mantra Abstrak

Kita akan membuat File: **DemoMantraAbstrak.java** yang didalamnya terdapat sebuah **`abstract class MantraAbstrak`** yang memiliki abstract method **`efek().`** Setiap mantra konkret harus mengimplementasikan method **`efek().`**

```java
import java.util.*;

abstract class MantraAbstrak {
    private String nama;

    public MantraAbstrak(String nama) {
        this.nama = nama;
    }

    // Abstract method: harus diimplementasikan oleh subclass
    public abstract void efek();

    // Method konkret
    public void ucapkan() {
        System.out.println("Mengucapkan mantra: " + nama);
    }
}

class Expelliarmus extends MantraAbstrak {
    public Expelliarmus() {
        super("Expelliarmus");
    }

    @Override
    public void efek() {
        System.out.println("Efek: Melucuti senjata lawan.");
    }
}

class ExpectoPatronum extends MantraAbstrak {
    public ExpectoPatronum() {
        super("Expecto Patronum");
    }

    @Override
    public void efek() {
        System.out.println("Efek: Memanggil Patronus untuk melawan Dementor.");
    }
}

public class DemoMantraAbstrak {
    public static void main(String[] args) {
        MantraAbstrak mantra1 = new Expelliarmus();
        MantraAbstrak mantra2 = new ExpectoPatronum();

        mantra1.ucapkan();
        mantra1.efek();

        mantra2.ucapkan();
        mantra2.efek();
    }
}
```

```
Mengucapkan mantra: Expelliarmus
Efek: Melucuti senjata lawan.
Mengucapkan mantra: Expecto Patronum
Efek: Memanggil Patronus untuk melawan Dementor.
```

Instruksi:

1. Jalankan kode di atas.
2. Tambahkan mantra baru, misalnya `Lumos` yang mewarisi `MantraAbstrak` dan implementasikan method `efek()`.

Pertanyaan:

- Apa yang terjadi jika kita mencoba membuat objek dari `MantraAbstrak` langsung? Coba dan jelaskan errornya.
- Mengapa kita perlu membuat abstract class? Apa keuntungannya?

Bagian 3: Kombinasi Polymorphism dan Abstract Class (40 menit)

3.1 Sistem Sihir yang Fleksibel

Kita akan membuat sistem sihir yang menggunakan *abstract class* dan *polymorphism*. Setiap penyihir bisa mengucapkan mantra, tetapi setiap mantra memiliki efek yang berbeda.

Instruksi:

1. Jalankan kode di bawah.
2. Modifikasi kode sehingga seorang penyihir bisa mengucapkan semua mantra yang sudah dipelajarinya dalam satu perintah (misalnya method **`ucapkanSemuaMantra()`**).

Pertanyaan:

- Bagaimana polymorphism dan abstract class bekerja sama dalam contoh ini?
- Apa yang akan terjadi jika kita menambahkan mantra baru tanpa mengubah kelas `Penyihir`? Apakah mudah? Mengapa?

Buatlah contoh kode di bawah dalam sebuah File: **DuniaSihir.java**

```java
import java.util.*;

import java.util.ArrayList;
import java.util.List;

abstract class Mantra {
    private String nama;

    public Mantra(String nama) {
        this.nama = nama;
    }

    public String getNama() {
        return nama;
    }

    // Abstract method untuk efek mantra
    public abstract void efek();
}

class Expelliarmus extends Mantra {
    public Expelliarmus() {
        super("Expelliarmus");
    }

    @Override
    public void efek() {
        System.out.println("Melucuti senjata lawan!");
    }
}

class ExpectoPatronum extends Mantra {
    public ExpectoPatronum() {
        super("Expecto Patronum");
    }

    @Override
    public void efek() {
        System.out.println("Memanggil Patronus!");
    }
}

class Lumos extends Mantra {
    public Lumos() {
        super("Lumos");
    }

    @Override
    public void efek() {
        System.out.println("Menerangi ujung tongkat!");
    }
}
```

```java
class Penyihir {
    private String nama;
    private List<Mantra> mantraList;

    public Penyihir(String nama) {
        this.nama = nama;
        this.mantraList = new ArrayList<>();
    }

    public void belajarMantra(Mantra mantra) {
        mantraList.add(mantra);
        System.out.println(nama + " belajar mantra: " + mantra.getNama());
    }

    public void ucapkanMantra(String namaMantra) {
        for (Mantra mantra : mantraList) {
            if (mantra.getNama().equals(namaMantra)) {
                System.out.println(nama + " mengucapkan: " + mantra.getNama());
                mantra.efek();
                return;
            }
        }
        System.out.println(nama + " belum belajar mantra " + namaMantra);
    }
}

public class DuniaSihir {
    public static void main(String[] args) {
        Penyihir harry = new Penyihir("Harry Potter");
        Mantra expelliarmus = new Expelliarmus();
        Mantra expectoPatronum = new ExpectoPatronum();

        harry.belajarMantra(expelliarmus);
        harry.belajarMantra(expectoPatronum);

        harry.ucapkanMantra("Expelliarmus");
        harry.ucapkanMantra("Expecto Patronum");
        harry.ucapkanMantra("Lumos"); // Harry belum belajar Lumos

        // Tambahkan mantra Lumos dan coba lagi
        Mantra lumos = new Lumos();
        harry.belajarMantra(lumos);
        harry.ucapkanMantra("Lumos");
    }
}
```

Output:

```
Harry Potter belajar mantra: Expelliarmus
Harry Potter belajar mantra: Expecto Patronum
Harry Potter mengucapkan: Expelliarmus
Melucuti senjata lawan!
Harry Potter mengucapkan: Expecto Patronum
Memanggil Patronus!
Harry Potter belum belajar mantra Lumos
Harry Potter belajar mantra: Lumos
Harry Potter mengucapkan: Lumos
Menerangi ujung tongkat!
```

TANTANGAN : Sistem Pertahanan Hogwarts

Hogwarts memiliki sistem pertahanan yang terdiri dari berbagai jenis mantra perlindungan. Setiap mantra perlindungan memiliki cara aktivasi dan efek yang berbeda.

Buatlah hierarki kelas berikut:

1. Abstract class `MantraPerlindungan` dengan method abstrak `aktifkan()` dan `nonaktifkan()`.
2. Beberapa kelas konkret yang mewarisi `MantraPerlindungan`, misalnya:
   - `Protego`: saat diaktifkan, mencetak `"Perisai Protego terbentuk!"`
   - `Fidelius`: saat diaktifkan, membutuhkan penyihir yang menjadi *secret keeper*.
   - `MantraAntiApparition`: mencegah *apparition* di area tertentu.
3. Kelas `SistemPertahanan` yang dapat menampung banyak mantra perlindungan dan memiliki method:
   - `tambahMantra(MantraPerlindungan mantra)`
   - `aktifkanSemuaMantra()`: mengaktifkan semua mantra yang ada.
   - `nonaktifkanSemuaMantra()`: menonaktifkan semua mantra.
4. Demonstrasikan dengan membuat beberapa objek mantra dan menambahkannya ke sistem pertahanan Hogwarts.

PENGUMPULAN:

1. Kumpulkan semua file code: `Mantra.java, DemoPenyihir.java, DemoMantraAbstrak.java` dan `DuniaSihir.java` serta code tantangan yang Anda buat dengan penamaan filename: **PBO_W5S3_NIM.zip**
2. **Wajib ada komentar pada setiap program** untuk menjelaskan baris kode atau blok logika atau penjelasan fungsi program.

Selamat berpetualang di dunia sihir dan coding! ☺

Selamat Mengerjakan